

Lab: Pipelining and Superscalar Concepts (Instructor's Guide)

Instructor Copy

October 16, 2025

Overview

This guide provides the solutions and detailed explanations for the student lab. The goal is for students to practically engage with processor architecture concepts by configuring and running simulations.

Part 1: In-Order Pipelined Processor - Solutions

Q1: Describe the configuration.

Answer: The configuration models a simple, scalar, in-order pipelined processor.

- **In-Order:** The `-issue:inorder true` flag forces the processor to issue instructions in the strict order they appear in the program.
- **Scalar (Single-Issue):** The `-decode:width 1` and `-issue:width 1` settings mean the processor can only fetch, decode, and issue one instruction per clock cycle.
- **Minimal Resources:** The `-res:*` settings provide only one of each type of functional unit.
- **Reservation Stations:** The `-ruu:size 8` and `-lsq:size 8` define the size of the instruction buffers.

Q2: What is the IPC of the `test-math` program using this configuration?

Answer: The IPC is **0.5071**. This is found in the `sim_configa.out` file.

Q3: Is the forwarding implemented?

Answer: Yes, forwarding is implemented.

- **Explanation:** A RAW hazard occurs when an instruction needs a result from a previous, unfinished instruction. Without forwarding, the pipeline would stall. With forwarding, the result is sent directly from the ALU output back to the ALU input for the next instruction.
 - **Evidence:** In the `config_a.trc` pipeline trace, students will find cases where a dependent instruction (e.g., `addu`) enters its Execute (EX) stage in the cycle immediately after the instruction it depends on (e.g., `sll`) completes its own EX stage. If the `sll` is in the MEM or WB stage while the `addu` is in EX, it proves forwarding occurred.
-

Part 2: Out-of-Order Superscalar Processor - Solutions

Q1: Describe the configuration.

Answer: This configuration models a simple, scalar, **out-of-order** processor. The key difference is the `-issue:inorder false` setting. This allows the processor to execute instructions whose operands are ready, even if they are not next in the program order. Other parameters remain the same.

Q2: What is the IPC of the test-math program using this configuration?

Answer: The IPC is **0.5184**, found in `sim.configb.out`.

Q3: Compared to part 1, what can you say about this configuration?

Answer: The out-of-order configuration (B) achieves a higher IPC (0.5184 vs. 0.5071) than the in-order configuration (A).

- **Explanation:** Out-of-order execution improves performance by hiding latency. When a long-latency instruction (like a cache miss) occurs, an in-order processor stalls. An out-of-order processor can work on later, independent instructions, increasing hardware utilization and overall instruction throughput (IPC).

Q4: The commit stage CT reorders the execution of instructions, explain why is it necessary?

Answer: The premise is slightly incorrect: the Commit (CT) stage does not reorder execution; it ensures **in-order commit** of results. This is necessary to maintain **precise exceptions**.

- **Explanation:** In an out-of-order machine, instructions can finish in any order. If a later instruction wrote its result to the register file before an earlier instruction caused a fault, the machine's state would be inconsistent, making it impossible for the OS to handle the exception correctly.
- **In-Order Commit:** The Re-Order Buffer (ROB) holds results temporarily. The commit stage writes these results to the permanent architectural state (register file, memory) in the original program order. This guarantees that if an exception occurs, the state is precise: all instructions before the faulting one are complete, and no instructions after it have modified the state.